

PROGRAMACIÓN II

Título del trabajo: *"Food Store – Sistema de Gestión de Pedidos de Comida (Consola + JDBC)"*

Alumnos:

- Cordero Campero Elisabeth (Comisión 5)
- Franco Analía Rocio (Comisión 5)
- Garcia Nadia Anahi (Comisión 5)

Profesor: Comisión 5: Renzo Sosa

Fecha de entrega: 15/06/2026

Índice de Contenido

Marco Teórico-----	3
1.1 Java como lenguaje de programación-----	3
1.2 Programación Orientada a Objetos-----	3
1.3 Encapsulamiento y Modificadores de acceso.-----	4
1.4 Herencia y clase abstracta base-----	4
1.5 Constructores, Polimorfismo y sobreescritura de métodos-----	4
1.6 Interfaces-----	5
1.7 Enumeraciones o Enums-----	5
1.8 Colecciones-----	5
1.9 CRUD-----	6
1.10 Manejo de excepciones -----	6
1.11 Validaciones -----	6
1.12 Bases de datos relacionales -----	6
Decisiones Técnicas y Arquitectura-----	7
Dificultades y Soluciones-----	9
Bibliografía-----	9
Link del video y repositorio de GITHUB-----	9

Sección 1

Marco Teórico

En el presente apartado explicaremos conceptos claves para la realización del trabajo : *“Food Store – Sistema de Gestión de Pedidos de Comida”*

1.1 Java como lenguaje de programación

Java es un lenguaje de programación de alto nivel desarrollado por Sun Microsystems en la década de 1990. Es fuertemente tipado y estático, lo cual lo distingue claramente entre los errores de compilación y los que ocurren durante la ejecución. Se caracteriza por ser orientado a objetos, lo que permite organizar el código a partir de clases y objetos, aplicando principios como **encapsulamiento, herencia y polimorfismo**.

Una de las características es su portabilidad, resumida en la filosofía "escribe una vez, ejecuta en cualquier lugar". Esto es posible gracias a la Máquina Virtual de Java (JVM), que actúa como una capa de abstracción entre el código Java y el sistema operativo.

Java también se destaca por su robustez, seguridad y versatilidad. Es utilizado en aplicaciones empresariales, desarrollo móvil para Android, aplicaciones web y sistemas embebidos.

En este proyecto se utiliza Java 21 para desarrollar una aplicación de consola basada en Programación Orientada a Objetos. El lenguaje permite representar las entidades principales del sistema Food Store, como usuarios, productos, categorías, pedidos y detalles de pedido, mediante clases relacionadas entre sí.

1.2 Programación Orientada a Objetos

Es un paradigma de programación que organiza el código en clases y objetos, inspirándose en cómo percibimos el mundo real.

- Clases: Una clase es una plantilla o molde que define atributos (características) y comportamientos (métodos) comunes de un conjunto de objetos y no ocupa memoria por sí sola. En el sistema Food Store, ejemplos de clases son Usuario, Producto, Categoría, Pedido y DetallePedido.
- Objeto: Un objeto es una instancia concreta de una clase y ocupa memoria durante la ejecución del programa. Cada objeto posee un estado, definido por el valor de sus atributos en un momento determinado, y una identidad propia que permite distinguirlo de otros objetos.
- Atributos: Los atributos representan las características o datos de una clase. Por ejemplo, un producto posee nombre, precio, descripción, stock, imagen y disponibilidad.
- Métodos: Los métodos representan las acciones o comportamientos que puede realizar una clase. Por ejemplo, la clase Pedido incluye métodos para agregar, buscar o eliminar detalles de pedido.

En Food Store, la Programación Orientada a Objetos se aplica mediante la creación de clases que representan los elementos principales del sistema. Por ejemplo, “Producto” representa los productos disponibles, “Categoría” agrupa los productos según su tipo. Estas clases se relacionan entre sí para representar el funcionamiento real del sistema. Por ejemplo, un producto se asocia con una categoría.

1.3 Encapsulamiento y Modificadores de acceso.

El encapsulamiento consiste en ocultar los detalles internos de un objeto y exponer solo una interfaz pública, esto se traduce en proteger los atributos de una clase y permitir su acceso o modificación a través de métodos. Existen métodos de accesos y modificación del valor de los atributos como los getter y setter, estos permiten controlar los datos internos de los objetos y evitar modificaciones incorrectas desde otras partes del programa.

En cuanto a los modificadores de objetos existen: `private` (accesible solo desde dentro de la misma clase), `public` (accesible desde cualquier parte del programa) y `protected` (accesible desde la misma clase, subclases y el mismo paquete).

En el sistema Food Store, el encapsulamiento se observa en el uso de atributos privados o protegidos y métodos públicos para acceder o modificar la información.

1.4 Herencia y clase abstracta base

La herencia permite crear nuevas clases (subclases o clases derivadas) de las ya existentes (superclase o clases bases), reutilizando atributos y métodos definidos en la superclase, promoviendo la reutilización de código y la extensibilidad. Esto permite establecer relaciones jerárquicas entre clases, donde la subclase hereda atributos y métodos de la superclase, pudiendo reutilizar o especializar su comportamiento.

En este proyecto se utiliza una clase abstracta llamada Base, de la cual heredan las entidades principales del sistema. Esta clase contiene atributos comunes como `id`, `eliminado` y `createdAt`. Gracias a esta estructura, clases como Categoría, Producto, Usuario, Pedido y DetallePedido no necesitan repetir esos atributos comunes.

1.5 Constructores, Polimorfismo y sobreescritura de métodos

Un constructor es un método especial que se ejecuta automáticamente cuando creas un nuevo objeto de una clase, su función principal es de inicializar los atributos del objeto recién creado. En el caso particular de java proporciona un constructor automático, si no se define ninguno con la particularidad de no recibir parámetros. De lo contrario al definir uno, java ya no proporciona el predeterminado. Los constructores pueden ser sobrecargados, permitiendo instanciar un objeto con diferentes parámetros, dando flexibilidad en la creación de objetos.

El polimorfismo permite que un mismo método pueda comportarse de diferentes maneras según la clase que lo implemente. Esto permite escribir código más flexible, facilitando su mantenimiento y la evolución del software. En este proyecto puede observarse mediante la sobreescritura de métodos como `toString()` y `equals()`.

El método `toString()` puede sobrescribirse en cada clase para mostrar la información de forma clara en los listados de consola.

En Food Store, el polimorfismo se observa principalmente a través de la interfaz Calculable, ya que la clase Pedido implementa el método calcularTotal() recorriendo sus detalles para obtener el total del pedido.

1.6 Interfaces

Una interfaz en Java es un tipo de referencia que define un contrato de comportamiento. Establece métodos que las clases deben implementar, permitiendo estandarizar acciones comunes sin definir necesariamente una implementación concreta.

En este proyecto se utiliza la interfaz Calculable, que establece el método calcularTotal(). Esta interfaz permite que distintas clases relacionadas con el proceso de venta puedan calcular importes de acuerdo con su propia lógica.

1.7 Enumeraciones o Enums

Un enum es un tipo especial de dato que permite representar un conjunto fijo de valores posibles, su función es estandarizar los valores posibles para evitar errores de entrada, mejorando la calidad y legibilidad del código.

Son “clases especiales” cuyas instancias (constantes) están fijadas dentro del enum. En Food Store se utilizan enumeraciones para representar valores cerrados y evitar errores de tipo. Por ejemplo, el enum Rol limita los tipos de usuario a valores como ADMIN y USUARIO. También se utilizan enums para representar el estado del pedido y la forma de pago. Esto permite que el sistema solo acepte opciones válidas y mejora la seguridad de los datos cargados.

1.8 Colecciones

Las colecciones en Java son estructuras dinámicas que permiten almacenar y gestionar grupos de objetos durante la ejecución del programa. En Food Store se utilizan colecciones como ArrayList para almacenar categorías, productos, usuarios y pedidos mientras el programa está en ejecución.

Además, las colecciones permiten representar relaciones uno a muchos entre clases. Una relación 1:N se produce cuando un objeto puede estar asociado con cero, uno o muchos objetos de otra clase. En Java, esta relación puede implementarse mediante un atributo de tipo colección dentro de una clase.

En el sistema Food Store, esto se observa en la clase Pedido, que contiene una lista de objetos DetallePedido. De esta manera, un pedido puede tener varios detalles, y cada detalle representa un producto solicitado con su cantidad correspondiente. Esta relación se implementa mediante una colección, permitiendo agregar, recorrer y calcular los subtotales de cada detalle.

También se utilizan recorridos sobre colecciones mediante estructuras como el bucle for-each, que permite procesar los elementos de una lista de forma más clara y segura, sin trabajar directamente con índices.

1.9 CRUD

CRUD es el acrónimo de Create, Read, Update y Delete, que representan las operaciones básicas de gestión de datos: crear, consultar, modificar y eliminar.

En Food Store, cada entidad principal cuenta con operaciones CRUD desde el menú de consola. Esto permite crear, listar, editar y eliminar categorías, productos, usuarios y pedidos.

Además, el sistema aplica el concepto de baja lógica. Esto significa que, al eliminar un registro, el objeto no se borra físicamente de la colección, sino que se modifica su estado interno mediante el atributo eliminado.

1.10 Manejo de excepciones

El manejo de excepciones permite gestionar errores y otros eventos excepcionales durante la ejecución del programa y evitar que la aplicación se cierre inesperadamente. En Java, esto se realiza mediante bloques try-catch.

En Food Store, las excepciones se utilizan para validar situaciones como intentar cargar un producto con precio negativo, ingresar stock inválido, seleccionar una entidad inexistente o agregar un detalle de pedido con cantidad menor o igual a cero.

Además del manejo general de errores, se implementan excepciones personalizadas, como `PrecioInvalidoException`, `StockInvalidoException`, `EntidadNoEncontradaException` y `MailDuplicadoException`. Estas excepciones permiten representar errores específicos del sistema y mostrar mensajes más claros al usuario.

1.11 Validaciones

Las validaciones son controles que se realizan antes de ejecutar una operación para asegurar que los datos ingresados sean correctos. En este sistema se aplican validaciones para evitar inconsistencias, como crear productos con precio o stock negativo, registrar usuarios con email repetido, crear pedidos sin usuario o cargar detalles con cantidad inválida.

En este proyecto, las validaciones se distribuyen entre las clases de servicio y los menús, manteniendo la lógica principal del negocio en los servicios y los controles de entrada en la interfaz de usuario.

1.12 Bases de datos relacionales

Una base de datos relacional organiza la información en tablas formadas por filas y columnas. Cada tabla representa una entidad, como usuarios, productos o pedidos, y se relaciona con otras mediante claves primarias y claves foráneas. En este trabajo, aunque se menciona el concepto de base de datos relacional y JDBC, el sistema Food Store no utiliza todavía una base de datos real. La información se

almacena en memoria mediante colecciones de Java, tal como indica la etapa actual del proyecto. Por eso, los datos solo existen mientras el programa está en ejecución y se pierden al cerrarlo.

SECCIÓN 2

Decisiones Técnicas y Arquitectura

Para abordar este proyecto, optamos por una arquitectura de Separación en Capas, lo que nos permitió organizar el código de forma lógica y escalable. La estructura se divide de la siguiente manera:

- Paquete entidades: Contiene el modelo de dominio basado en el UML. Aplicamos Herencia mediante una clase Base para atributos comunes (ID, fecha de creación) e implementamos la interfaz Calculable para centralizar la lógica en la clase Pedido, que recorre sus detalles y calcula el total del pedido.
- Paquete servicios: Aquí centralizamos la lógica de negocio y las validaciones. Los servicios actúan como puente entre la interfaz y el almacenamiento, asegurando que los datos cumplan con las reglas (como stock disponible o mails únicos) antes de ser guardados.
- Paquete interfazUsuario: Para evitar un Main saturado de código, movimos toda la interacción con el operador a este paquete.

Implementamos una Clase Abstracta MenuBase para estandarizar el comportamiento de todos los menús, aplicando el principio DRY (Don't Repeat Yourself) para el manejo de excepciones y bucles de control.

- Paquete config: En la clase Listas se centraliza el almacenamiento temporal de las entidades mediante colecciones en memoria. También se administra la generación correlativa de los identificadores utilizados durante la ejecución del sistema.

Captura de pantalla del sistema funcionando:

```

run:

--- SISTEMA FOOD STORE ---
1. Gestión de Categorías
2. Gestión de Productos
3. Gestión de Usuarios
4. Gestión de Pedidos
0. Salir
Seleccione una opción: 1

--- GESTIÓN DE CATEGORÍAS ---
1. Crear Categoría
2. Listar Categorías
3. Editar Categoría
4. Eliminar Categoría
0. Volver
Seleccione una opción: 1
Nombre de la categoría: Pastas
Descripción: Pastas caseras con salsas a eleccion
¡Categoría creada con éxito!

--- GESTIÓN DE CATEGORÍAS ---
1. Crear Categoría
2. Listar Categorías
3. Editar Categoría
4. Eliminar Categoría
0. Volver
Seleccione una opción: 0

--- SISTEMA FOOD STORE ---
1. Gestión de Categorías
2. Gestión de Productos
3. Gestión de Usuarios
4. Gestión de Pedidos
0. Salir
  
```

```

--- SISTEMA FOOD STORE ---
1. Gestión de Categorías
2. Gestión de Productos
3. Gestión de Usuarios
4. Gestión de Pedidos
0. Salir
Seleccione una opción: 4

--- GESTIÓN DE PEDIDOS ---
1. Crear Nuevo Pedido
2. Listar Pedidos
3. Editar Estado/Forma de Pago
4. Eliminar Pedido
0. Volver
Seleccione una opción: 1

--- LISTA DE USUARIOS ACTIVOS ---
ID: 1 - Juan Perez (juan@mail.com) - Rol: USUARIO
ID: 2 - Maria Gomez (maria@mail.com) - Rol: ADMIN
ID: 3 - Carlos Lopez (carlos@mail.com) - Rol: USUARIO
Seleccione el ID del Usuario para ingresar el pedido: 4
Error: Usuario no encontrado.

--- GESTIÓN DE PEDIDOS ---
1. Crear Nuevo Pedido
2. Listar Pedidos
3. Editar Estado/Forma de Pago
4. Eliminar Pedido
0. Volver
Seleccione una opción: 2

--- LISTADO DE PEDIDOS ---
¿Desea filtrar por un usuario específico? (S/N): n
  
```

```

--- LISTADO DE PEDIDOS ---
¿Desea filtrar por un usuario específico? (S/N): n

--- RESULTADO DE LA BÚSQUEDA ---
=====
PEDIDO ID: 1 | FECHA: 2026-05-30
CLIENTE: Juan Perez
CONTACTO: juan@mail.com
ESTADO: PENDIENTE | PAGO: TARJETA
DETALLES DEL PEDIDO:
* - Muzzarella x2 | Subtotal: $24000,00
* - Coca-Cola 1.5L x1 | Subtotal: $2500,00

>>> TOTAL DEL PEDIDO: $26500,00
=====

PEDIDO ID: 2 | FECHA: 2026-05-30
CLIENTE: Carlos Lopez
CONTACTO: carlos@mail.com
ESTADO: PENDIENTE | PAGO: EFECTIVO
DETALLES DEL PEDIDO:
* - Tiramisu x3 | Subtotal: $13500,00

>>> TOTAL DEL PEDIDO: $13500,00
=====

--- GESTIÓN DE PEDIDOS ---
1. Crear Nuevo Pedido
2. Listar Pedidos
3. Editar Estado/Forma de Pago
4. Eliminar Pedido
0. Volver
Seleccione una opción:
  
```


SECCIÓN 3

Dificultades y Soluciones

Durante el desarrollo del sistema, nos enfrentamos a varios desafíos técnicos que logramos resolver mediante investigación y trabajo en equipo:

La primera dificultad fue la robustez de las entradas de datos. Al principio, el programa se cerraba si el usuario ingresaba letras en campos numéricos como el Precio o el ID.

Para eso implementamos bloques try-catch específicos para la excepción `NumberFormatException` dentro de bucles do-while. Esto nos permitió capturar el error, informar al usuario con un mensaje amigable y darle la oportunidad de reintentar sin perder su progreso.

Otro desafío era la consistencia en la carga de Pedidos. Notamos que si un pedido fallaba a mitad de camino (por ejemplo, por falta de stock en el tercer producto), los productos anteriores ya habían sido restados del inventario, dejando datos erróneos.

Entonces aplicamos un mecanismo manual de reversión de cambios. El sistema registra temporalmente los productos y cantidades modificados y, si ocurre un error, restaura el stock original antes de cancelar el pedido.

El último desafío fue la coordinación con Git y GitHub. Al ser una de nuestras primeras experiencias colaborativas con Git, la gestión de ramas y los conflictos de código fueron un reto inicial. Por ello establecimos un flujo de trabajo basado en 6 ramas y el uso obligatorio de Pull Requests. Esto nos obligó a revisar el código de cada compañera antes de integrarlo a la rama principal, garantizando que el proyecto siempre estuviera en un estado estable y funcional.

SECCIÓN 4

Bibliografía

Material de Cátedra (s.f.). Introducción a Java . UTN.

Material de Cátedra (s.f.).U3-A1-Fundamentos-de-la-Programación-Orientada-a-Objetos-en-Java. UTN.

Material de Cátedra (s.f.).U4-A1-Profundizando-en-Clases-y-Objetos-en-Java. UTN.

Material de Cátedra (s.f.).Contenido Teórico Actividad 1. UTN.

Oracle. (2023). The Java Language Specification, Java SE 21 Edition.

<https://docs.oracle.com/javase/specs/jls/se21/html/jls-1.html>

SECCIÓN 5

Link del video y repositorio de GITHUB

Link Video Youtube

<https://youtu.be/Z83WDUwQ2n0>

Link Repositorio GitHub

https://github.com/elisabethcordero/TPI_Prog2_Final